

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI – 602105
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Debugging Test

Course Code:	DSA05	Course Title:	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	Assessment:	Tool 3 – Debugging Test
Weightage:		Total Marks:	25 Marks
CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BL6)			
Student Name:	Sania Herbert	Reg. No.:	192324062
Section / Batch:	2023/AI&DS	Date of Submission:	04.08.2026
Faculty In-charge:	Dr. B.SHYAMALA GOWRI	Project Mode:	Individual Submission

Code of Conduct

I Sania Herbert(192324062) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: Sania

Error Identification(5)	Code Correction(8)	SQL Query Accuracy(5)	Explanation and Justification(4)	Program Execution and Output(3)	Total (25 Marks)

Q3. Debug the MySQL Connection `import mysql.connector`

```

connection=mysql.connector.connect(host="localhost",user="root"password="root",database="c
ollege"
)
cursor=connection.cursor()
cursor.execute("SELECT * FROM student")
for i in cursor:print(i)

```

Task

- Identify every syntax and indentation error.
- Rewrite the corrected program.
- Explain why the original program fails.

SOLUTION:

1. Error Identification

S.No	Error	Explanation
1	Missing comma after user="root"	Causes a Python syntax error.
2	Incorrect connect() argument separation	Arguments must be separated by commas.
3	Cursor and connection are not closed	May lead to resource leakage.
4	No exception handling	Program crashes if database connection fails.
5	Poor formatting/indentation	Reduces readability and maintainability.

2. Code Correction

```
import mysql.connector
```

```
try:
```

```
    connection = mysql.connector.connect(
```

```
        host="localhost",
```

```
        user="root",
```

```
        password="root",
```

```
        database="college"
```

```
    )
```

```
    cursor = connection.cursor()
```

```

cursor.execute("SELECT * FROM student")

for row in cursor:

    print(row)

cursor.close()

connection.close()

except mysql.connector.Error as err:

    print("Error:", err)

```

3. SQL Query Accuracy

SQL Query Used

SELECT * FROM student;

Justification

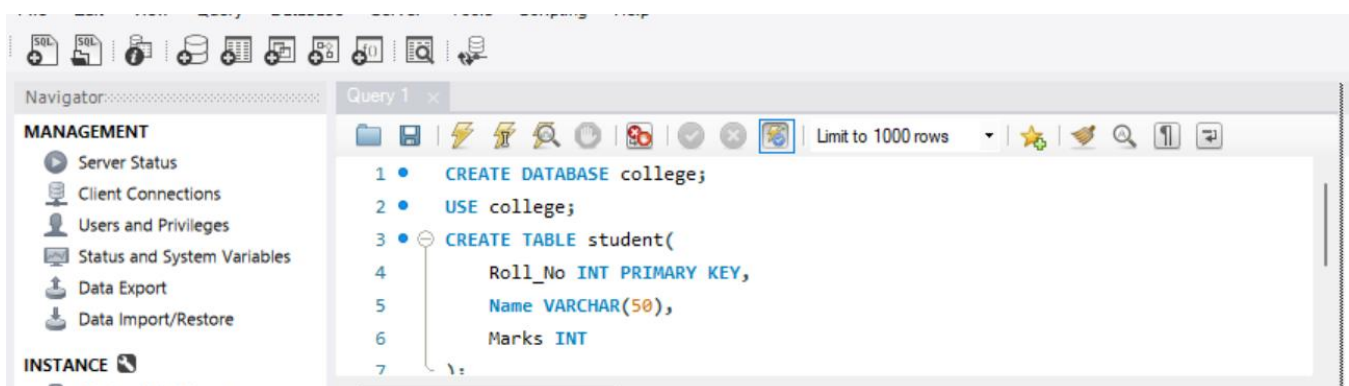
- Retrieves all records from the student table.
- SQL syntax is correct.
- Executed using cursor.execute().

4. Explanation and Justification

- The original program fails because a comma is missing between user="root" and password="root".
- Python treats this as invalid syntax and stops execution before connecting to MySQL.
- Closing the cursor and connection releases database resources.
- Exception handling makes the program robust by displaying meaningful error messages instead of crashing.

5. Program Execution and Output (3 Marks)

Assume the **student** table contains:



```

7      -  })
8  ●    INSERT INTO student VALUES
9      (1,'Ram',90),
10     (2,'Priya',85),
11     (3,'Kiran',95);
12  ●    SELECT * FROM student;

```

OUTPUT

	Roll_No	Name	Marks
▶	1	Ram	90
	2	Priya	85
	3	Kiran	95
✱	NULL	NULL	NULL

Q4. Debug the PostgreSQL Program import psycopg2

```

conn=psycopg2.connect(dbname="college",user="postgres",password="admin "
)
cursor=conn.cursor()
cursor.execute("INSERT INTO Student VALUES(%s,%s,%s)",(1,"Ram"))
conn.commit()
cursor.execute("SELECT * FROM Student")
print(cursor.fetchone())
conn.close()

```

Task

- Identify all errors.
- Rewrite the corrected program.
- Explain the effect of parameter mismatch.

1. Error Identification

S.No	Error	Explanation
1	Three placeholders (%s) but only two values supplied	Parameter mismatch error.
2	Missing third value (Marks)	INSERT statement requires all column values.

S.No	Error	Explanation
3	Cursor is not closed	Database resources remain open.
4	No exception handling	Program terminates if an error occurs.
5	Improper resource management	Connection should be closed after cursor.

2. Code Correction

```

import psycopg2

try:

    conn = psycopg2.connect(

        dbname="college",

        user="postgres",

        password="admin"

    )

    cursor = conn.cursor()

    cursor.execute(

        "INSERT INTO Student VALUES(%s, %s, %s)",

        (1, "Ram", 90)

    )

    conn.commit()

    cursor.execute("SELECT * FROM Student")

    print(cursor.fetchone())

    cursor.close()

    conn.close()

except Exception as e:

    print("Error:", e)

```

3. SQL Query Accuracy (5 Marks)

INSERT Query

```
INSERT INTO Student VALUES (1, 'Ram', 90);
```

SELECT Query

```
SELECT * FROM Student;
```

Justification

- Number of placeholders matches the number of supplied values.
- Queries follow correct PostgreSQL syntax.
- `conn.commit()` permanently saves the inserted record.

4. Explanation and Justification (4 Marks)

- The original query contains **three placeholders** but only **two values** are passed.
- PostgreSQL cannot map the supplied values to all placeholders, causing a parameter mismatch error.
- Adding the missing value (90) resolves the issue.
- Closing the cursor and connection ensures proper database resource management.

5. Program Execution and Output (3 Marks)

```
import psycopg2

conn = psycopg2.connect(

    dbname="college",

    user="postgres",

    password="admin"

)

cursor = conn.cursor()

cursor.execute(

    "INSERT INTO Student VALUES(%s,%s,%s)",

    (1,"Ram",90)

)
```

```
conn.commit()

cursor.execute("SELECT * FROM Student")

for row in cursor.fetchall():

    print(row)

cursor.close()

conn.close()
```

Assume the **Student** table is initially empty.

After execution, the inserted record is:

OUTPUT:

id	name	marks
----	------	-------

1	Ram	90
---	-----	----

(1 row)